Python 程式設計

第七章：物件導向程式設計 (OOP)

Prof. Nien-Lin Hsueh

Department of Information Engineering
Feng Chia University

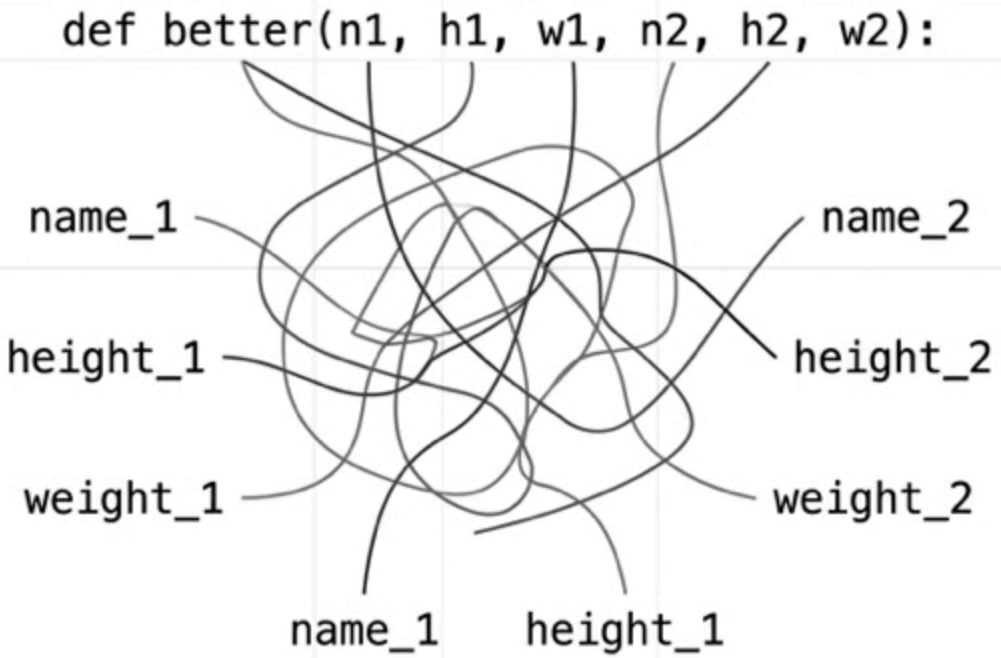
本章學習重點

- **7.1 類別與物件 (Class & Object)**
  - 物件導向 vs 功能導向、藍圖與實例概念
  - 類別宣告、物件封裝設計與 Data Classes (Python 3.7+)
- **7.2 屬性與封裝 (Attribute & Encapsulation)**
  - 物件屬性 vs 類別屬性、私有屬性與名稱修飾 (Name Mangling)
  - 特性與裝飾器 (`@property`、`@setter`)、動態屬性
- **7.3 繼承與多型 (Inheritance & Polymorphism)**
  - 基礎類別與衍生類別、方法覆寫 (Override) 與 `super()` 呼叫
- **7.4 特殊方法與抽象類別 (Special Methods & Abstract Classes)**
  - 雙底線特殊方法 (dunder methods)、自訂有理數 Rational 類別
  - 抽象類別宣告 (`abc.ABCMeta`) 與抽象方法 (`@abstractmethod`)

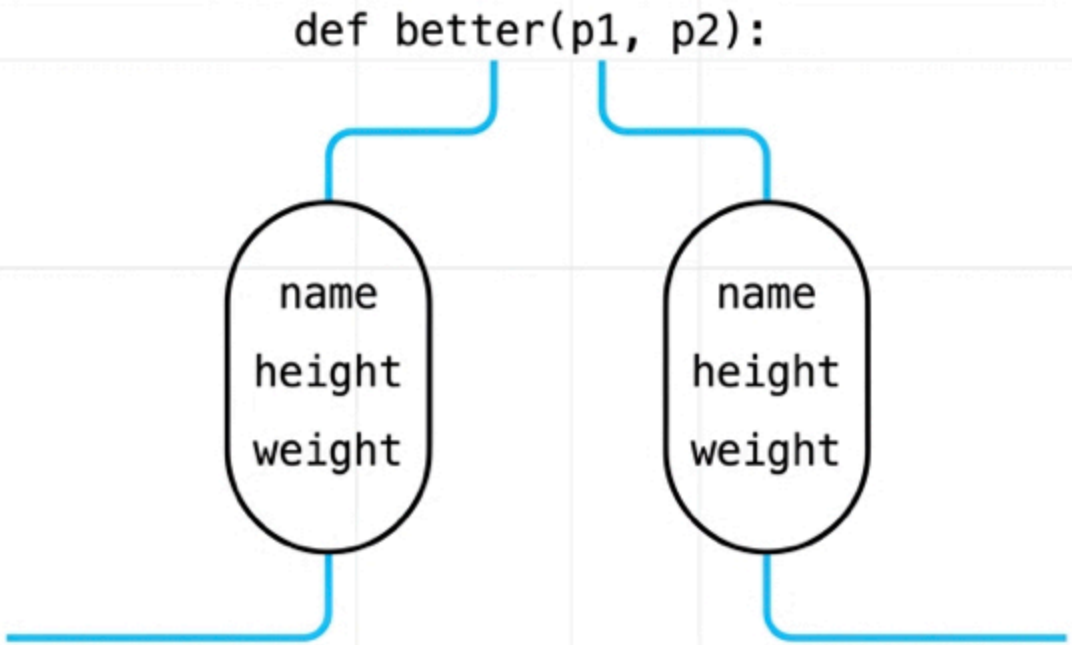
7.1 類別與物件 (Class & Object)

The Anatomy of Spaghetti Code

The Problem



The Antidote



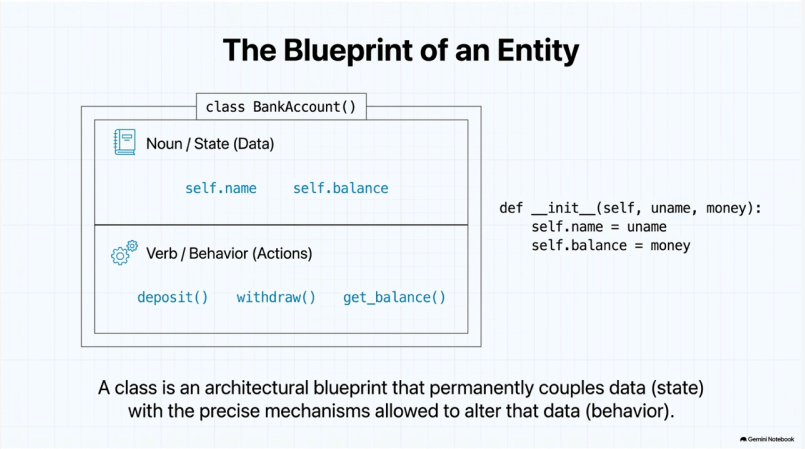
Object-Oriented Programming (OOP) exists to manage system complexity by binding related data into singular, transportable entities.

Gemini Notebook

物件導向設計 (Object-Oriented Design)

- **功能導向**：將大系統切割成小函式，以函式為最小單元來解決複雜性。
- **物件導向**：以**物件**為最小單元。物件內部**封裝了資料以及對應的操作方法**。

- **高內聚 (High Cohesion)**：資料與其行為緊密包裝。
- **降低複雜度**：透過物件邊界與接口，讓程式結構更加清晰、容易組裝與維護。



類別的宣告與物件生成

- **類別 (Class)**：定義物件特徵與行為的「藍圖/模板」。
- **物件 (Object/Instance)**：根據藍圖實際建立出來的「實體」。

```
# 類別的宣告
class class_name():
    def __init__(self, x, y):
        self.x = x  # 初始化物件變數
        self.y = y

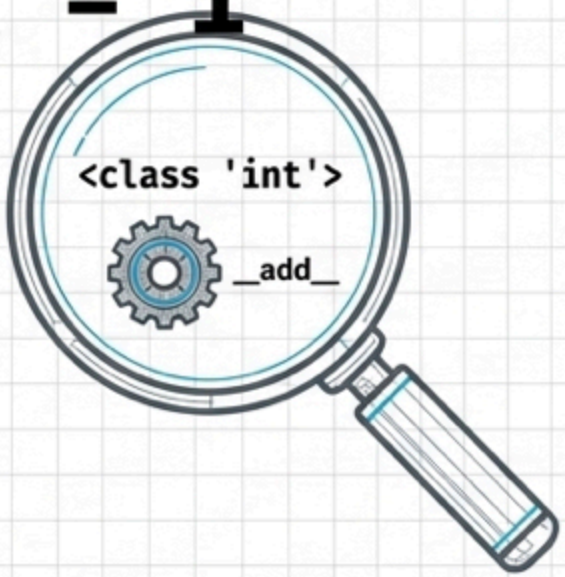
    def getX(self):
        return self.x

# 物件的生成 (實例化)
obj = class_name(10, 20)
print(obj.getX()) # 輸出: 10
```

Objects All The Way Down

The Integer

```
x = 1
```



The List

```
y = []
```



The Custom Entity

```
z = Currency('USD', 100)
```



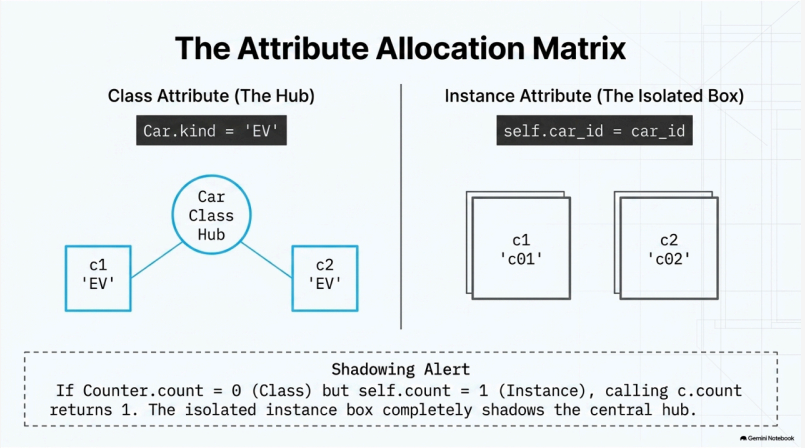
Python does not differentiate between built-in types and your code. By mastering classes, encapsulation, properties, and magic methods, you are not just writing scripts on top of Python—you are extending the language itself.

Gemini Notebook

Python 中一切皆物件

- 在 Python 中，所有的資料型態（如 `int`, `float`, `list`, `dict`）都是類別實作的物件。

- **以 `list` 物件為例**：
  - **內部資料**：儲存的元素集合。
  - **方法**：提供 `append()`、`extend()`、`insert()` 等方法來操作內部的資料。



封裝的好處：以 People 類別為例

- 透過物件將「資料」與「行為」封裝，可使程式碼更加簡潔，避免參數過多的混亂設計。

```
class People:
    def __init__(self, name, tall, weight):
        self.name = name
        self.tall = tall
        self.weight = weight

    def get_bmi(self):
        return self.weight / (self.tall ** 2)

# 物件導向設計：只需傳入兩個 People 物件
def better(p1, p2):
    return p1.name if p1.get_bmi() < p2.get_bmi() else p2.name

# 傳統設計：參數過多，不夠簡潔
# def better(n1, h1, w1, n2, h2, w2):
```

現代 Python：資料類別 (Data Classes)

- 自 Python 3.7 起引入 `@dataclass`，專門簡化「單純儲存資料」類別的宣告，自動生成建構子、字串表達及比較方法。

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    student_id: str
    grades: list[int]

# 自動生成 __init__
s1 = Student('Nick', 'S9201201', [90, 72, 100])
s2 = Student('Nick', 'S9201201', [90, 72, 100])

# 自動生成友善的 __repr__
print(s1)  # 輸出: Student(name='Nick', student_id='S9201201', grades=[90, 72, 100])

# 自動生成 __eq__ (比較內容而非記憶體地址)
```

現代 Python：Self 型態提示 (Python 3.11+)

- 當類別的方法會返回**該物件本身**（例如鏈式呼叫）時，在型態提示中可使用 `typing.Self`：

```
from typing import Self

class Book:
    def __init__(self, title: str):
        self.title = title

    def rename(self, new_title: str) -> Self:
        self.title = new_title
        return self  # 回傳物件本身，型態提示為 Self

b1 = Book("Python 101")
b1.rename("Python Advanced").rename("Python Expert") # 鏈式呼叫
```

物件相關輔助函式

- Python 提供了數個內建函式用來查詢與操作物件：
  - `type(obj)`：返回物件所屬的類別。
  - `dir(obj)`：列出物件的所有方法和屬性。
  - `id(obj)`：取得物件的唯一識別值（記憶體位置）。
  - `hasattr(obj, name)`：檢查屬性是否存在於物件中。
  - `getattr(obj, name, default)`：安全地獲取屬性值。
  - `callable(obj)`：檢查物件是否可被呼叫（如函數或類別）。

```
class BMI:
    def __init__(self, w, h):
        self.weight = w
        self.height = h

b = BMI(70, 1.7)
print(hasattr(b, 'weight'))  # True
print(getattr(b, 'age', 18)) # 18 (安全取得屬性)
```

7.2 屬性與封裝 (Attribute & Encapsulation)

物件屬性 vs 類別屬性

- **物件屬性 (實體屬性)**：在 `__init__` 中以 `self.xxx` 定義，每個實例獨立擁有。
- **類別屬性**：直接在類別區塊中定義，所有實例共享。

```
class Car:
    kind = '燃油車'                # 類別變數 (所有 Car 共用)

    def __init__(self, car_id):
        self.car_id = car_id     # 物件變數 (各別 Car 獨立)

c1 = Car('c1')
c2 = Car('c2')
print(c1.kind, c2.kind) # 燃油車 燃油車

Car.kind = '電動車'          # 修改類別變數，所有實例的 kind 皆改變
print(c1.kind, c2.kind) # 電動車 電動車
```

Concept Check Question (CCQ 1)

給定下列 Python 類別定義：

```
class Counter:
    count = 0  # 類別屬性 (Class Attribute)
    def __init__(self):
        self.count = 1  # 實例屬性 (Instance Attribute)

c = Counter()
print(Counter.count, c.count)
```

請問程式執行的輸出結果為何？



- **A.** `0 0`
- **B.** `0 1`
- **C.** `1 1`
- **D.** 引發 `AttributeError`

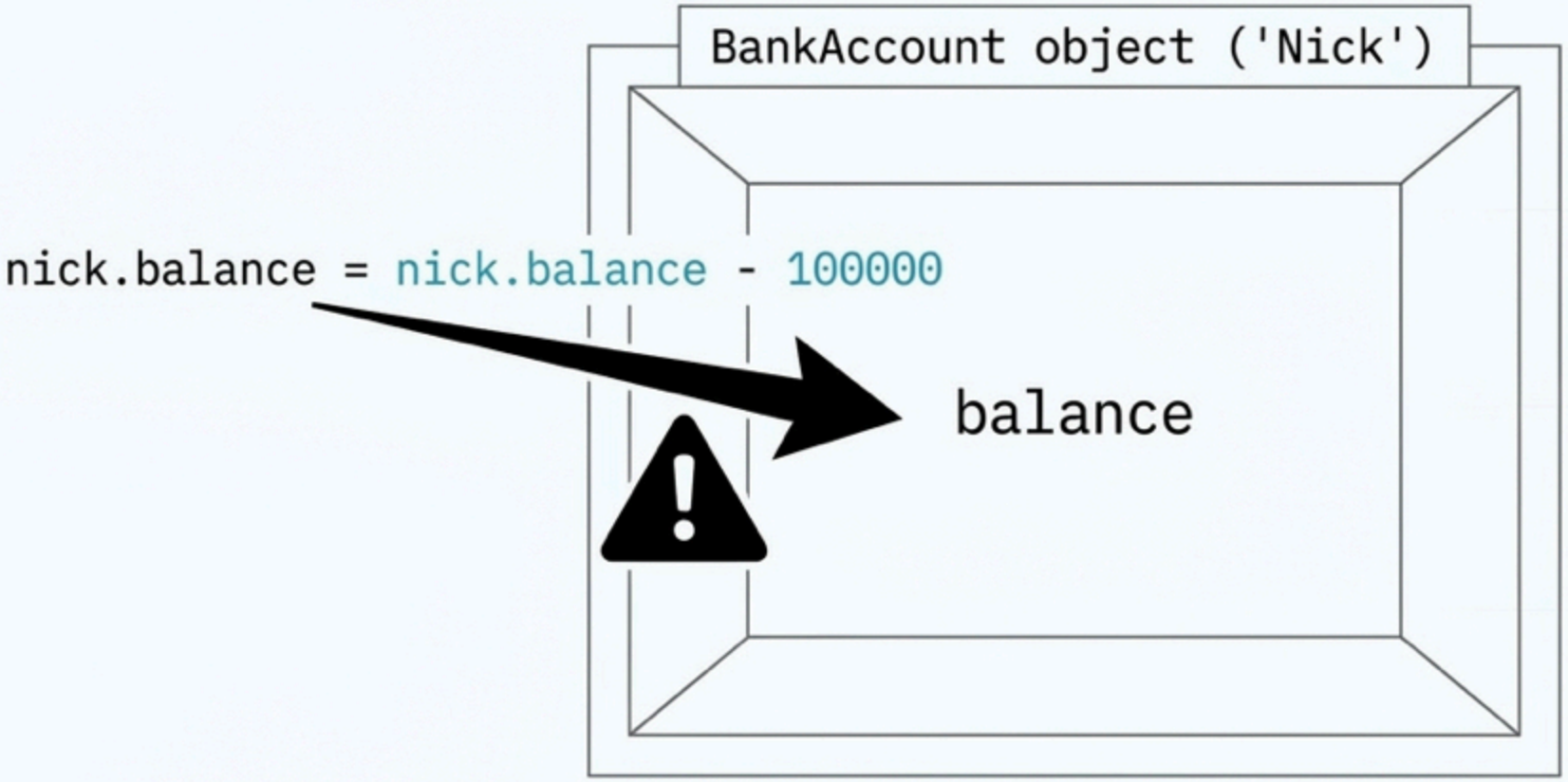
CCQ 1 - 答案與解析

正確答案：B

- **解析**：
  - `Counter.count` 存取的是定義在類別層級的**類別屬性**，其值為 `0`。它被所有實例共用，但不能透過實例進行直接覆寫（除非特別指定）。
  - `c.count` 存取的是實例 `c` 初始化時在 `__init__` 中建立的**實例屬性**，其值為 `1`。
  - 實例屬性與類別屬性同名時，實例屬性會**遮蔽 (shadow)** 類別屬性，因此存取 `c.count` 會優先返回實例屬性的值 `1`。



The Vulnerability of Open State



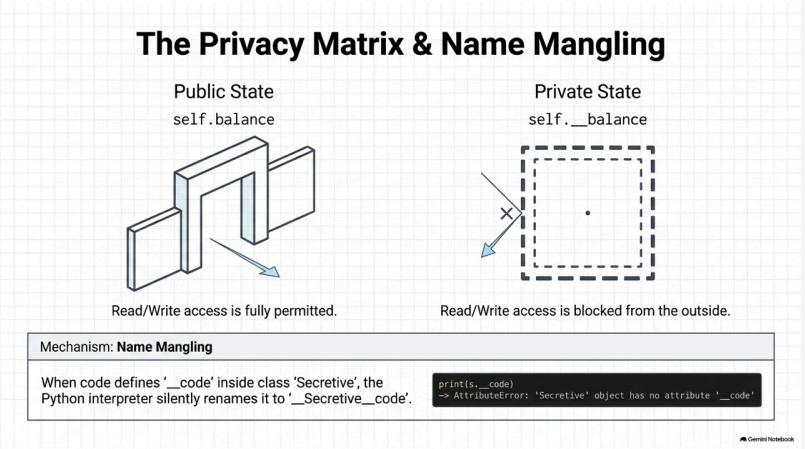
Public attributes allow external code to bypass the object's defined behaviors (methods), violating the architectural integrity of the system.

Gemini Notebook

私有屬性 (Private Attributes)

- 如果屬性公開，外面的程式可以任意修改，這可能造成物件內部的邏輯或狀態混亂（例如銀行餘額變為負值）。

- **雙底線命名**：在屬性名稱前加上雙底線 `__`，將其宣告為私有。
- 外部無法直接透過 `obj.__balance` 存取私有變數。



私有屬性與保護機制

- 透過 Getter / Setter 方法間接存取私有屬性，可在內部加入商業邏輯檢查（例如存款不可為負，提款需確認餘額）：

```
class BankAccount:
    def __init__(self, uname, money):
        self.name = uname
        self.__balance = money  # 私有屬性

    def withdraw(self, money):
        # 提款前先檢查餘額安全性
        if self.__balance - money < 0:
            raise Exception('餘額不足')
        self.__balance -= money

    def get_balance(self):
        return self.__balance

nick = BankAccount('Nick', 10000)
# nick.__balance = 0 # 錯誤：無法直接存取
```

Concept Check Question (CCQ 2)

下列程式碼執行時會發生什麼事？

```
class Secretive:
    def __init__(self):
        self.__code = 42

s = Secretive()
print(s.__code)
```



- **A.** 正常執行，印出 42
- **B.** 正常執行，印出 None
- **C.** 引發 AttributeError
- **D.** 引發 NameError

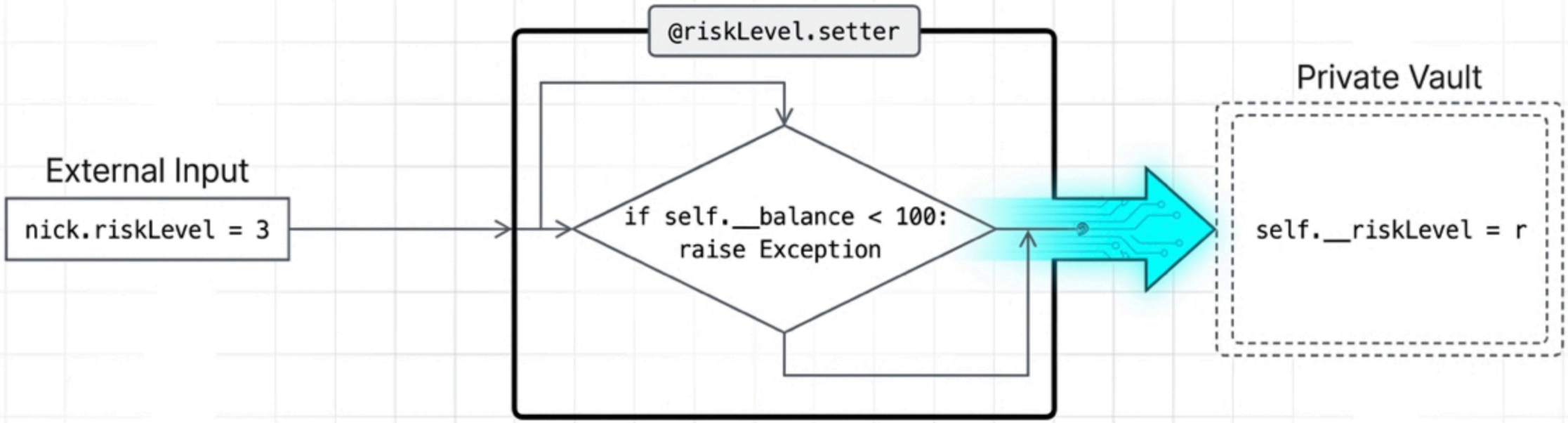
CCQ 2 - 答案與解析

正確答案：C

- **解析**：
  - 在 Python 中，以兩個底線開頭但不用兩個底線結尾的屬性名稱（例如 `__code`），會觸發 **名稱修飾 (Name Mangling)** 機制。
  - 直譯器會自動將這個屬性重新命名為 `_ClassName__attributeName`（即 `_Secretive__code`），以避免在繼承關係中意外衝突。
  - 因此，在外部直接透過 `s.__code` 存取該屬性時，會因為找不到此名稱而引發 `AttributeError: 'Secretive' object has no attribute '__code'`。



The Security Checkpoint



The @property decorator allows developers to write code that looks like simple variable assignment while secretly executing complex validation logic behind the scenes.

Gemini Notebook

使用 @property 特性

- 傳統 Getter/Setter 需要繁瑣的 `obj.get_x()`，而 `@property` 裝飾器可以讓存取看起來像一般屬性，但底層仍受方法保護。

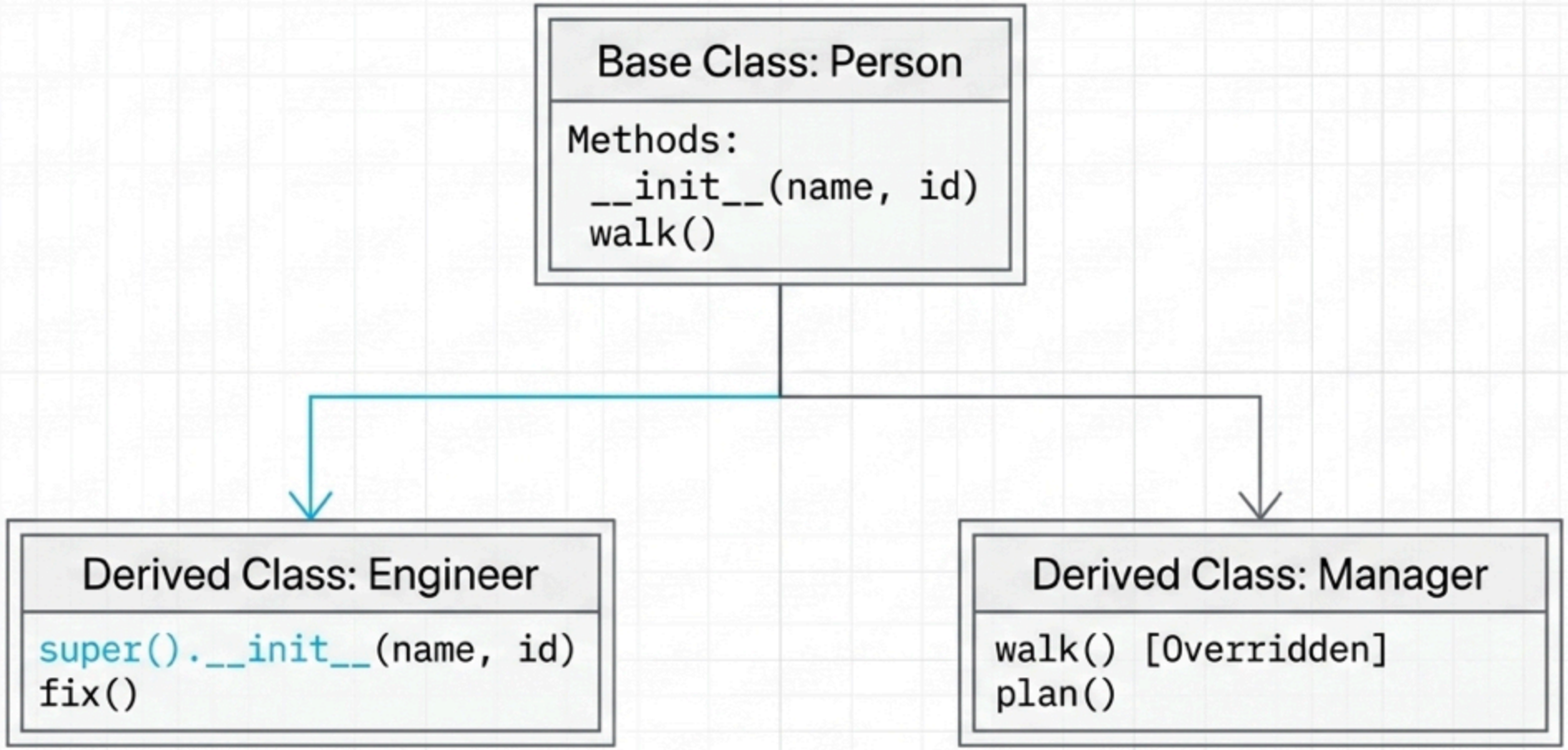
```
class BankAccount:
    def __init__(self, uname, money):
        self.name = uname
        self.__balance = money
        self.__riskLevel = 0

    @property
    def riskLevel(self): # getter 宣告，直接以屬性名命名
        level = {0: 'not set', 1: 'Low', 2: 'Medium', 3: 'High'}
        return level[self.__riskLevel]

    @riskLevel.setter
    def riskLevel(self, r): # setter 宣告，形式為 @屬性名.setter
        if self.__balance < 100:
            raise Exception("存款不足，無法設定此風險等級")
        self.__riskLevel = r
```

7.3 繼承與多型 (Inheritance & Polymorphism)

Architectural Taxonomy



Inheritance is not just about code reuse; it is about establishing a rigorous hierarchy where specialized entities inherit, extend, and override fundamental behaviors.

Gemini Notebook

繼承 (Inheritance)

- 繼承允許子類別 (衍生類別) 取得父類別 (基礎類別) 的屬性與功能，提高程式碼重用性。

```
class Person:
    def __init__(self, name, id):
        self.name = name
        self.id = id
    def walk(self):
        print(self.name, 'is walking')

# Engineer 繼承 Person
class Engineer(Person):
    def __init__(self, name, id, skill):
        super().__init__(name, id) # 呼叫父類別建構子
        self.skill = skill         # 子類別新增屬性
    def fix(self):                 # 子類別新增方法
        print(self.name, 'is fixing bug')
```

方法覆寫 (Override) 與多型

- 子類別可以重新定義父類別中同名的方法，以展現其專屬行為，這即是**方法覆寫 (Override)**：

```
class Manager(Person):
    # Override 父類別的方法
    def walk(self):
        print(self.name, 'is walking and thinking')
    def plan(self):
        print(self.name, 'is planning for new project')

Jack = Engineer('Jack', 'S1234', 'Python')
Mary = Manager('Mary', 'S5678')
Nick = Person('Nick', 'R9012')

group = [Jack, Mary, Nick]
for p in group:
    p.walk() # 展現多型特徵，各別物件執行對應版本的 walk()
```

Concept Check Question (CCQ 3)

給定下列繼承關係程式碼：

```
class Parent:
    def __init__(self):
        self.val = 10

class Child(Parent):
    def __init__(self):
        self.val = 20

c = Child()
print(c.val)
```



請問印出的結果為何？

- **A.** 10
- **B.** 20

CCQ 3 - 答案與解析

正確答案：B

- **解析**：
  - 子類別 `Child` 定義了自己的建構子 `__init__`，這會直接**覆寫 (override)** 父類別 `Parent` 的建構子。
  - 當我們實例化 `Child()` 時，只有子類別的建構子會被執行，其中 `self.val` 被設定為 `20`。
  - 因為子類別建構子內沒有呼叫 `super().__init__()`，所以父類別建構子沒有被執行（但 `self.val` 已經在子類別中建立並賦值），故 `c.val` 回傳的值是 `20`。



7.4 特殊方法與抽象類別 (Special Methods & Abstract Classes)

Bridging Custom and Native Architecture

Block A (Clunky)

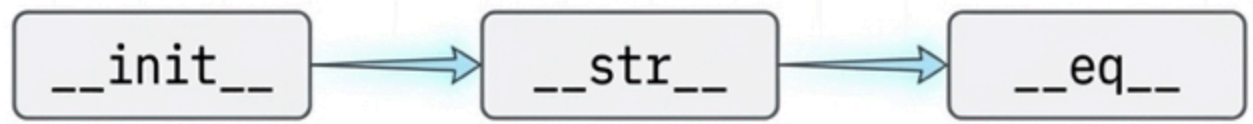
```
x = Rational(1, 2)
y = Rational(2, 3)
result = x.add(y)
```

Block B (Elegant)

```
x = Rational(1, 2)
y = Rational(2, 3)
result = x + y
```

The ultimate goal of object-oriented Python is to build custom entities that interact with operators exactly like built-in integers or lists.

System Overrides (Magic Methods)



Gemini Notebook

Python 特殊方法 (Magic Methods)

- 特殊方法（或稱魔法方法，Dunder Methods）是以雙底線開頭與結尾的方法。
- 它們定義了物件與 Python 內建運算子（如 `+`, `-`, `==`, `len()`, `str()`）的行為。

特殊方法	呼叫時機
`__init__(self, ...)`	創建對象（構造函數）時
`__str__(self)`	使用 `str(obj)` 或 `print(obj)` 時
`__repr__(self)`	互動式命令列中輸入物件名稱時
`__add__(self, other)`	使用 `+` 加法運算時
`__sub__(self, other)`	使用 `-` 減法運算時
`__eq__(self, other)`	使用 `==` 等於運算時

特殊方法應用：有理數 Rational 類別

- 自訂 Rational (分數) 類別，透過實作特殊方法，使其能與內建數值般進行運算：

```
class Rational:
    def __init__(self, n, d):
        self.numer = n
        self.denom = d

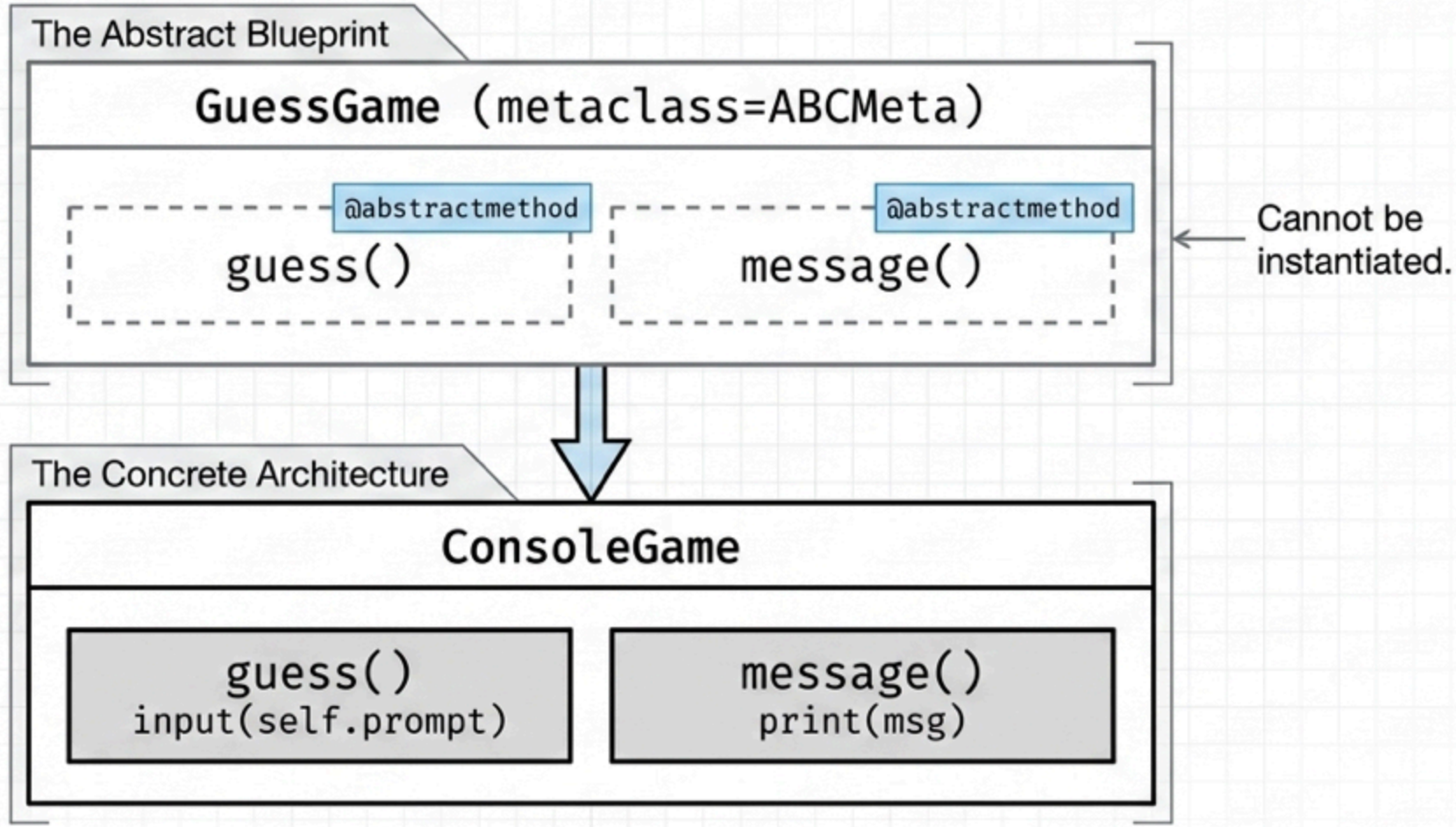
    def __str__(self): # 定義 print(obj) 的格式
        return f"{self.numer}/{self.denom}"

    def __add__(self, that): # 定義 + 運算
        return Rational(self.numer * that.denom + that.numer * self.denom,
                        self.denom * that.denom)

    def __eq__(self, that): # 定義 == 運算
        return self.numer * that.denom == that.numer * self.denom

a = Rational(1, 2)
b = Rational(2, 4)
```

Forcing Implementation Blueprints



Abstract classes dictate what must exist, forcing derived classes to dictate how it operates.

Gemini Notebook

抽象類別 (Abstract Class)

- 抽象類別是用來定義規格（方法介面）的類別，不能直接被實例化，需由子類別繼承並實現其抽象方法。
- Python 中使用 abc 模組的 ABCMeta 與 @abstractmethod 宣告。

```
from abc import ABCMeta, abstractmethod

class GuessGame(metaclass=ABCMeta): # 宣告為抽象類別
    @abstractmethod
    def message(self, msg):
        pass

    @abstractmethod
    def guess(self):
        pass

    def go(self): # 大部流程已定義，細節 guess 與 message 留給子類別
        self.message("Welcome!")
        # 遊戲骨幹邏輯...
```

抽象類別實作與限制

- 子類別必須完全實作所有的抽象方法，否則該子類別亦被視為抽象而無法實例化：

```
class ConsoleGame(GuessGame):
    def __init__(self):
        self.prompt = "輸入數字："

    def message(self, msg): # 實作抽象方法
        print(msg)

    def guess(self): # 實作抽象方法
        return int(input(self.prompt))

# game_err = GuessGame() # ERROR: 抽象類別無法被實例化！
game = ConsoleGame()     # 正確：子類別已實作所有抽象方法
game.go()
```